

Sensor de distância por infravermelho

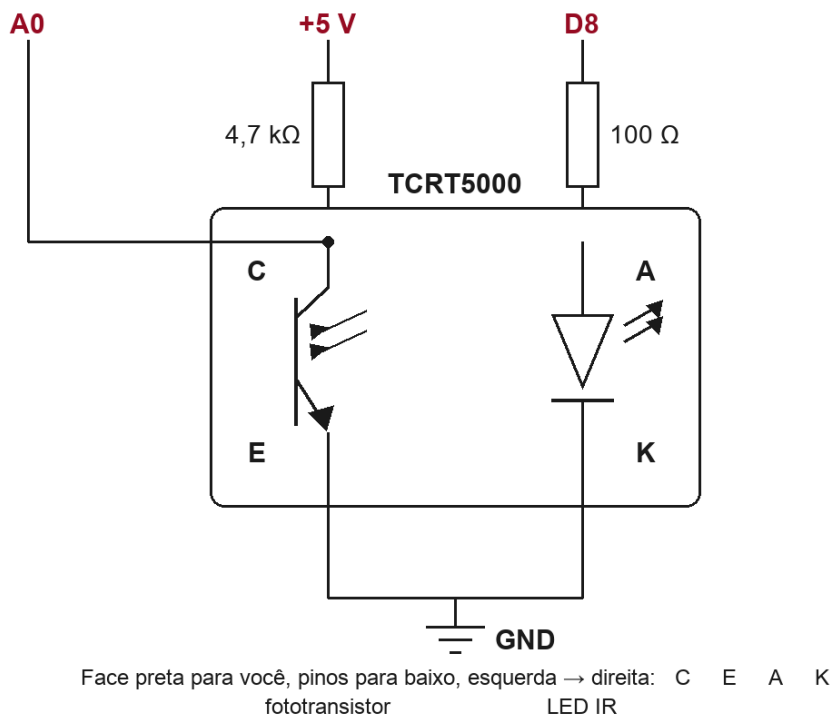
TCRT5000 na protoboard · relatório até 6 de outubro de 2026

1. O sensor

Um sensor de distância estima a separação até um alvo sem encostar nele. Na faixa do infravermelho isso se faz com um emissor e um detector. O fotodiodo transforma os fótons que absorve numa corrente pequena. O fototransistor usa o mesmo efeito na base de um transistor: a luz gera os portadores, e o ganho do transistor entrega uma corrente maior, mais fácil de ler com um resistor, embora a resposta seja mais lenta. O TCRT5000 reúne os dois no mesmo invólucro. O diodo emite em torno de 950 nm; o fototransistor tem um filtro que barra a maior parte da luz visível. A emissão sai, reflete no alvo e uma parte volta ao detector. Quanto mais perto e mais claro o alvo, maior a corrente no coletor. O que o conversor analógico do Arduino entrega é essa corrente, vista como tensão e convertida num inteiro de 0 a 1023. A distância, em milímetros, só aparece depois, quando um modelo inverte essa curva.

2. Montagem

O trabalho é em equipe, na protoboard, com a Mega alimentada pelos bornes de 5 V e GND. Cada grupo recebe um TCRT5000, um resistor de 4,7 k Ω e um de 100 Ω . O alvo é um cartão branco, com uma régua fixa para marcar a distância até a face do sensor.



O 4,7 k Ω liga o 5 V ao coletor e ao pino A0. O 100 Ω sai do D8 e entra no anodo.

O resistor de 100 Ω limita a corrente do emissor em cerca de 38 mA enquanto o D8 está em nível alto. O de 4,7 k Ω é o pull-up do coletor. Sem luz infravermelha, o fototransistor fica aberto e A0 permanece perto de 5 V. Quando a luz refletida chega, o transistor conduz e a tensão em A0 cai. A reflexão, portanto, aparece como uma queda da leitura analógica. O D8 existe para que o programa ligue e desligue o emissor; com o resistor ligado direto ao 5 V o diodo ficaria aceso o tempo todo e

não haveria como separar a reflexão da luz da sala.

Os quatro terminais, com a face preta virada para vocês e os pinos para baixo, seguem da esquerda para a direita: coletor, emissor, anodo e catodo. A câmera do celular mostra o LED infravermelho como uma mancha violeta quando o D8 está em HIGH. Se o A0 não se mexer com o cartão na frente do sensor, coletor e emissor estão trocados.

3. Leitura no Arduino

O sketch abaixo vale no Arduino IDE e no PlatformIO da Mega (exemplos/tcrt5000). O ciclo tem quatro passos. O D8 vai a HIGH e o programa espera o tempo ligado; analogRead(A0) guarda a leitura com o emissor aceso. O D8 volta a LOW, o programa espera o tempo desligado e lê de novo. A segunda leitura é a luz ambiente: lâmpada, janela, o que o filtro não barrou. O sinal usado daqui para a frente é a diferença entre as duas, limitada a zero quando a conta daria negativo.

$$\text{sinal} = \max(0, \text{adc_off} - \text{adc_on})$$

Os dois tempos começam em 1000 ms. Nesse regime o LED acende e apaga de forma visível, e cada linha da serial corresponde a um ciclo completo. A corrente de cerca de 38 mA pede que o pino não fique parado em HIGH: o próprio loop o desliga ao fim de cada leitura.

exemplos/tcrt5000/src/main.cpp

```
#include <Arduino.h>

// LED infravermelho no D8, coletor do fototransistor no A0.
// Comecem com 1 s ligado e 1 s desligado. Depois reduzam os dois.
const int PIN_LED = 8;
const int PIN_ADC = A0;
unsigned long t_on_ms = 1000;
unsigned long t_off_ms = 1000;

void setup() {
  pinMode(PIN_LED, OUTPUT);
  digitalWrite(PIN_LED, LOW);
  Serial.begin(115200);
}

void loop() {
  digitalWrite(PIN_LED, HIGH);
  delay(t_on_ms);
  int adc_on = analogRead(PIN_ADC);

  digitalWrite(PIN_LED, LOW);
  delay(t_off_ms);
  int adc_off = analogRead(PIN_ADC);

  int sinal = adc_off - adc_on;
  if (sinal < 0) sinal = 0;
  Serial.println(sinal);
}
```

4. Medidas na serial

Gravem o sketch e abram o monitor em 115200. Cada linha é um inteiro: o sinal daquele ciclo. Com o cartão longe, ou sem cartão, os dois analógicos ficam parecidos e o número oscila perto de zero. Ao aproximar o cartão branco, adc_on cai, a diferença cresce e o número sobe. Essa subida é a confirmação de que emissor, detector e resistores estão no lugar. Se o valor permanecer em zero com o cartão encostado, voltem à polaridade dos pinos antes de seguir.

```
pio run -t upload && pio device monitor
```

5. Gráfico em tempo real

O monitor e o script Python não podem abrir a mesma porta ao mesmo tempo. Fechem o monitor. O script lê a serial com pyserial e desenha as últimas amostras com matplotlib. A porta da Mega aparece em pio device list; no Linux costuma ser /dev/ttyACM0 e no Mac, /dev/cu.usbmodem... . Se

faltar biblioteca: pip install pyserial matplotlib.

```
python3 plot_serial.py /dev/ttyACM0
```

Com o cartão parado, a linha treme: isso é o ruído da leitura. Aproximem e afastem o cartão devagar. A curva que aparece na tela é a mesma grandeza da serial, agora fácil de ver. Ainda não é distância. É o sinal, em contagens do conversor.

exemplos/plot_serial.py

```
#!/usr/bin/env python3
"""Plota ao vivo a serial da Mega.

Uma coluna (o sinal cru): desenha contagens.
Duas colunas sinal,mm: desenha a distância em milímetros.

python3 plot_serial.py /dev/ttyACM0
"""

import sys
import time
from collections import deque

import matplotlib.pyplot as plt
import serial

porta = sys.argv[1] if len(sys.argv) > 1 else "/dev/ttyACM0"
ser = serial.Serial(porta, 115200, timeout=1)
time.sleep(2) # a Mega reinicia ao abrir a USB

n = 0
xs, ys = deque(maxlen=200), deque(maxlen=200)
modo = None

plt.ion()
fig, ax = plt.subplots()
(linha,) = ax.plot([], [], color="#960B22", lw=1.5)
ax.set_xlabel("amostra")
fig.tight_layout()

def amostra(bruto):
    if bruto.isdigit():
        return "cru", float(bruto)
    partes = bruto.split(",")
    if len(partes) == 2:
        try:
            return "mm", float(partes[1])
        except ValueError:
            return None
    return None

try:
    while True:
        bruto = ser.readline().decode(errors="ignore").strip()
        lido = amostra(bruto)
        if lido is None:
            continue
        kind, valor = lido
        if modo is None:
            modo = kind
            if modo == "mm":
                ax.set_ylabel("distância (mm)")
            else:
                ax.set_ylabel("sinal (contagens)")
        if kind != modo:
            continue
        n += 1
        xs.append(n)
        ys.append(valor)
        print(bruto, flush=True)
        linha.set_data(xs, ys)
        ax.relim()
        ax.autoscale_view()
        plt.pause(0.001)
except KeyboardInterrupt:
```

```
pass
finally:
    ser.close()
```

6. Caracterização até 200 mm

Caracterizar o sensor é anotar o sinal em distâncias conhecidas. Fixem o TCRT5000, mantenham o cartão branco perpendicular à face e leiam a régua da face do sensor até o papel. O intervalo vai até 200 mm. Quantas distâncias usar, e quantas leituras repetir em cada uma, é decisão do grupo: mais repetições mostram o ruído e estabilizam a média; pontos de menos deixam a curva frouxa. Incluam também uma série sem alvo, que é o fundo de ruído.

Anotem, para cada posição, a distância em milímetros e os sinais que saíram na serial. Passem isso para uma planilha. A coluna da média do sinal, em cada distância, é o que entra no modelo. A coluna do desvio é o ruído daquela distância.

Dois trechos não entram no ajuste. Muito perto, `adc_on` cola em zero: o fototransistor saturou, e a folha de dados do TCRT5000 ainda mostra um pico de resposta por volta de 2,5 mm, região em que o sinal deixa de cair de forma monótona. Perto dos 200 mm, a média pode encostar no valor sem alvo. O modelo usa o intervalo em que o sinal ainda diminui quando a distância aumenta. Se isso acabar antes dos 200 mm, o último ponto acima do fundo é o alcance desta montagem, e esse fato já é um resultado.

7. Modelo inverso, na planilha

A luz que volta de um alvo difuso cai, em primeira aproximação, com o quadrado da distância. Enquanto o sensor não satura e o sinal ainda se destaca do ruído, a curva medida tem a forma

$$S(d) = \alpha / d^2$$

S é a média do sinal em contagens e d é a distância da régua. O modelo direto vai da distância para o sinal. O que interessa na prática é o caminho inverso: dado o inteiro que acabou de sair do conversor, estimar a distância. Invertendo a expressão, $d = \sqrt{\alpha} / \sqrt{S}$. Isso é uma reta na variável $z = 1/\sqrt{S}$. Um segundo ganho desloca a reta e absorve o que o inverso puro não descreve: o zero da régua, o tamanho finito do alvo e o residual de ambiente.

$$d = g_0 + g_1 / \sqrt{S}$$

g_0 e g_1 são os ganhos, em milímetros e em milímetros vezes a raiz da contagem. Na planilha, criem a coluna $z = 1/\sqrt{S}$. A regressão linear da distância da régua contra z dá os dois números: g_1 é a inclinação e g_0 é o intercepto. No LibreOffice e no Google Planilhas, INCLINAÇÃO e INTERCEPÇÃO calculam isso; no Excel, SLOPE e INTERCEPT. Em seguida, a coluna $d_est = g_0 + g_1 \cdot z$ é a distância que o modelo devolve. Façam dois gráficos: o sinal médio contra a distância da régua, e d_est contra a distância da régua, com a reta $y = x$ por cima. O afastamento dessa reta é o erro do modelo, em milímetros. Guardem g_0 e g_1 . Eles vão para o Arduino.

8. Desafio: o modelo no Arduino

O desafio final é fazer o próprio sketch devolver a distância. Com os ganhos da planilha, alterem o loop para imprimir, na mesma linha, o sinal cru e a distância em milímetros, separados por vírgula: `sinal,mm`. A conta é $d = g_0 + g_1/\sqrt{S}$, e só vale com $S > 0$. Quando o sinal for zero, não dividam por zero: imprimam um valor que vocês consigam justificar. Gravem de novo.

O mesmo `plot_serial.py`, ao receber duas colunas, desenha a segunda, em milímetros. Aproximem e afastem o cartão e confrontem o número da tela com a régua. Essa curva ao vivo, junto com o gráfico da planilha, é a calibração funcionando no microcontrolador.

9. Até onde o sensor enxerga

A caracterização convida uma pergunta direta. Duzentos milímetros é a maior distância que este sensor alcança? O alcance é o ponto em que o sinal se confunde com o fundo, e ele depende da corrente no emissor, do tempo em que o LED permanece aceso, da refletância do alvo e da luz da sala. A folha de dados, com corrente menor e um alvo de laboratório, concentra a resposta útil em poucos centímetros, com pico perto de 2,5 mm. Com o cartão branco e 1 s de acionamento, a curva de vocês pode ir além dos 200 mm ou morrer bem antes. O valor que importa é o que a régua mostrar quando a média deixar de se separar da leitura sem alvo.

A pergunta seguinte é o que acontece se esse tempo diminui. Alterem `t_on_ms` e `t_off_ms` no sketch, gravem de novo e repitam três distâncias que já tenham sinal claro em 1 s. Uma sequência razoável é 1000 ms, 100 ms, 10 ms e 1 ms, com os dois tempos iguais. O sinal em cada distância deve cair, e a distância máxima também.

A razão é física, e vale para o relatório. Quando o pino sobe, a corrente no diodo não produz a potência óptica final no mesmo instante. Os portadores ocupam a junção e a emissão cresce ao longo de dezenas ou centenas de microssegundos, até o regime em que a corrente direta já se estabeleceu. No fototransistor o caminho é o inverso: os fótons geram portadores na base, a corrente de coletor cresce com o ganho do transistor e, depois que o LED apaga, a carga ainda armazenada sustenta corrente por mais um intervalo. Se o tempo ligado é curto, o emissor não chega à potência e o detector não chega à corrente correspondente. A diferença `adc_off - adc_on` encolhe. O ruído da leitura continua da mesma ordem, então a distância em que o sinal se mistura ao fundo fica menor. O intervalo de 1 s deixa os dois dispositivos em regime quase contínuo. É nele que esta montagem, com o resistor de 100 Ω e este alvo, mostra o maior alcance. É também o regime em que o LED dissipa potência durante todo o tempo em que está aceso.

10. Relatório

O relatório é em equipe e fecha este estudo. A entrega é na terça-feira, 6 de outubro de 2026. O texto descreve o que foi medido, apresenta a planilha com os ganhos `g0` e `g1` e o erro em milímetros, e discute o alcance. Na discussão, expliquem o que acontece fisicamente quando o LED é ligado e desligado por um intervalo curto, usando as medidas de vocês e não só o argumento geral da seção anterior. Fotografem a montagem, o arranjo da régua com o alvo e as telas dos gráficos.

Documentem as escolhas: quantas distâncias e repetições usaram, o que entrou no ajuste, o que foi descartado e por quê, e quais tempos de acionamento foram comparados.

- Foto da protoboard e do arranjo régua e cartão.
- Planilha com as medidas, as médias, `z`, `g0`, `g1` e `d_est`.
- Gráfico do sinal até 200 mm e gráfico da distância estimada, em mm.
- Sketch que imprime sinal e milímetros, e a curva ao vivo em mm.
- Até onde o sinal se separou do fundo, e se 200 mm era o limite.
- O que a redução do tempo fez com o sinal e com o alcance, e por quê.

A planilha e o sketch final acompanham o relatório.